

Kafka Training - Trainer Playbook

Pre-Training Checklist

- ❑ Docker Desktop running
- ❑ Run scripts/start-kafka.bat (wait 30 seconds)
- ❑ Run scripts/create-topics.bat
- ❑ Run mvn clean package -DskipTests
- ❑ Open <http://localhost:8080> in browser
- ❑ Open 2 terminal windows
- ❑ Slides ready (if applicable)

DAY 1 PLAYBOOK

MODULE 1: Why Kafka & When to Use It (60 minutes)

Learning Objectives

- Understand Kafka vs traditional messaging
- Know when to choose Kafka
- Understand payment event use cases

Setup (5 min)

Terminal 1: Ready for running demos
Browser: <http://localhost:8080> open

Sub-Module 1.1: Messaging vs Streaming (15 min)

Trainer Script:

"Let's start with a question. How does your payment service currently communicate with your fraud detection service?" (Wait for answers: HTTP calls, RabbitMQ, etc.)

"The problem with direct calls and traditional queues is they create TIGHT COUPLING. Let me show you."

Draw on whiteboard:

```
Traditional: PaymentSvc → [Queue] → FraudSvc ONLY
                ↑
                ↑ LedgerSvc CAN'T get it
                ↑ AuditSvc CAN'T get it

Kafka:         PaymentSvc → [Topic] → FraudSvc (Group A)
                → LedgerSvc (Group B)
                → AuditSvc (Group C)
                → AnalyticsSvc (Group D)
```

Key points to cover:

- Traditional MQ: message consumed = message gone
- Kafka: message written = multiple groups can consume
- Kafka retains messages (configurable, default 7 days)
- Payment replay: run new fraud model against all old payments

Sub-Module 1.2: Kafka Ecosystem Walkthrough (15 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day1Module1_WhyKafka.KafkaEcosystemDemo"
```

Point out in logs:

1. Demo 1 output: "PROBLEM 1-5" - these are real problems students face
2. Demo 2 output: The Kafka solution diagram
3. Demo 3 output: Watch for "Event produced" lines - real messages!

Switch to Kafka UI:

- Show <http://localhost:8080>
- Click "Brokers" → show 3 brokers online
- Click "Topics" → show payment-created with 3 partitions
- Click "payment-created" → click "Messages" → show the events produced

Discussion question:

"What other use cases in your organization could benefit from event streaming?"

Sub-Module 1.3: Payment Event Use Cases (15 min)

Whiteboard exercise: Draw the payment processing flow with students:

```
[Customer App] → Payment Gateway → [Kafka]
                                     |— Fraud Detection (real-time, <100ms)
                                     |— Ledger Update
                                     |— Audit Log
                                     |— Analytics Dashboard
                                     |— Regulatory Reporting
```

Key points:

- "All these services are INDEPENDENT - one failure doesn't cascade"
- "Adding a new service = just subscribe to the topic, zero changes upstream"
- "Replay capability = run new compliance checks on historical data"

Sub-Module 1.4: Architecture Overview (15 min)

Show in Kafka UI:

- Topics list: payment-created, fraud-result, etc.
- Cluster overview: 3 brokers healthy

Q&A:

- "Why 3 brokers?" → Replication, fault tolerance, no single point of failure
 - "Why use Kafka for payments specifically?" → Throughput, ordering, replay, durability
-

MODULE 2: Kafka Architecture (60 minutes)

Learning Objectives

- Understand brokers, topics, partitions
 - Understand replication and ISR
 - Understand leader election
-

Sub-Module 2.1: Broker Architecture Demo (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module2_Architecture.BrokerArchitectureDemo"
```

Walk through the output section by section:

Demo 1 - Cluster Brokers:

"See the 3 broker IDs: 1, 2, 3. One of them is the CONTROLLER. The controller manages partition leadership."

Demo 2 - Topic Structure:

"Look at payment-created. It has 3 partitions. Each partition has a LEADER and REPLICAS."

Demo 3 - ISR:

"ISR = In-Sync Replicas. Only brokers that are caught up with the leader are in the ISR. The leader waits for ALL ISR members to confirm before acking acks=all writes."

Live demo - Kill a broker:

```
docker stop kafka-broker-1
```

- Watch Kafka UI → broker goes red
- Re-run the describe command to see ISR change
- Start broker back up:

```
docker start kafka-broker-1
```

Sub-Module 2.2: Partition & Replication Demo (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module2_Architecture.PartitionReplicationDemo"
```

Demo 1 - No Key (Round Robin):

"Watch the partition numbers - they change for each message. This gives us load balancing but NO ordering guarantee."

Demo 2 - With Key:

"Watch CUST-AAA - all its messages go to the SAME partition! This is the ORDERING guarantee. All payments for one customer are in sequence."

Show in Kafka UI:

- Go to payment-partition-demo topic
- Show Messages tab
- Filter by partition to see distribution

Sub-Module 2.3: Architecture Q&A (20 min)

Common questions to answer:

Q: "What happens if we run out of disk on a broker?" A: "Kafka will stop writing to that broker. ISR shrinks. If $ISR < min.insync.replicas$, writes fail."

Q: "Can we increase partitions after creation?" A: "YES but cannot decrease. Add partitions with `kafka-topics --alter`. But existing key → partition mappings change!"

Q: "How many partitions should we create?" A: "Rule of thumb: $\max(\text{expected_consumers} \times 2, 10)$. For payments: 12-24 is typical."

MODULE 3: Producers & Consumers (60 minutes)

Learning Objectives

- Implement reliable producers
- Implement consumer groups
- Manage offsets correctly
- Understand delivery semantics

Sub-Module 3.1: Producer Demo (20 min)

Run Demo:

```
mvn exec:java -
  Dexec.mainClass="com.kafka.training.Day1Module3_ProducersConsumers.PaymentProducer"
```

Walk through each demo:

Demo 1 - Sync Producer:

"See the latency numbers? Each message waits for broker confirmation. Safe but SEQUENTIAL - one at a time."

Demo 2 - Async with Callback:

"Now they're sent in parallel! Notice 'Queued' appears before 'Callback'. The callback fires after Kafka confirms the batch."

Demo 3 - Batch Producer (1000 messages):

"Watch the throughput number. Even on localhost we get thousands of messages per second. Production: millions/sec."

Key config to highlight in code:

```
acks=all          → payment safety
linger.ms=5       → batch accumulation
enable.idempotence → no duplicates
```

Sub-Module 3.2: Consumer Demo (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module3_ProducersConsumers.PaymentConsumer"
```

Demo 1 - Auto Commit:

"Simple but risky. If we crash after poll() returns but before we finish processing, we LOSE the message."

Demo 2 - Manual Commit:

"We process FIRST, commit AFTER. If we crash during processing, we reprocess. This gives us AT-LEAST-ONCE." "For payments: we need idempotency! Store paymentId in DB, check before processing to avoid double payments."

Demo 3 - Seek to Beginning:

"REPLAY! This is Kafka's killer feature vs traditional MQ. Watch it read all historical messages from offset 0."

Sub-Module 3.3: Consumer Groups Demo (15 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module3_ProducersConsumers.ConsumerGroupDemo"
```

Key observations to point out:

- Phase 1: Consumer-1 gets all 3 partitions
- Phase 2: "REBALANCE" in logs when Consumer-2 joins!
- Phase 3: Each consumer gets exactly 1 partition

Show in Kafka UI:

- Consumer Groups section
- Watch partition assignments change during rebalance

Key insight:

"This is HOW you scale Kafka consumers. Need more throughput? Add another consumer instance. Kubernetes + Kafka = scale payment processing on demand!"

Sub-Module 3.4: Offset Management (5 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module3_ProducersConsumers.OffsetManagementDemo"
```

Key point for Demo 3:

"offsetsForTimes() - give me all messages from 30 minutes ago. No need to know exact offsets. Timestamp-based replay. Perfect for: 'Replay all payments from the last hour'."

MODULE 4: Reliability & Ordering (60 minutes)

Learning Objectives

- Configure ACKs correctly
 - Use idempotent producers
 - Design key-based partitioning
 - Understand ordering guarantees
-

Sub-Module 4.1: ACK Demo (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module4_Reliability.AcknowledgementDemo"
```

Walk through each ACK mode:

acks=0:

"Fastest. Producer doesn't wait at all. NEVER use for financial data. Would you deposit money and walk away without a receipt?"

acks=1:

"Leader confirms. Followers sync ASYNCHRONOUSLY after. Small window where leader could crash before replication. Still risky for payment systems."

acks=all:

"ALL ISR brokers confirm. Slowest but GUARANTEED. This is the only acceptable setting for payment events. You get your receipt only after the bank vault is secured."

Demo 4 - Latency comparison:

"Look at the actual milliseconds. The difference is usually 5-20ms. For payments, that's acceptable latency for safety."

Sub-Module 4.2: Idempotent Producer (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module4_Reliability.IdempotentProducerDemo"
```

Demo 1 - The problem:

"Without idempotence: network timeout → retry → DUPLICATE PAYMENT! Customer sees \$500 charged twice. Support ticket. Angry customer."

Demo 2 - The solution:

"enable.idempotence=true adds a sequence number. Broker says 'I already have sequence 42 from producer PID-5, ignoring this duplicate.'"

Demo 3 - Transactions:

"For atomic operations across topics. Payment event + Ledger update = ONE TRANSACTION. If either fails, BOTH are rolled back."

Show code highlight:

```
props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, "true");
// Automatically sets: acks=all, retries=MAX_VALUE
```

Sub-Module 4.3: Key-Based Partitioning (20 min)

Run Demo:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day1Module4_Reliability.KeyBasedPartitioningDemo"
```

Demo 1 - Ordering explained:

"Draw this: Partition 0: CREATED(1) → APPROVED(2) → SETTLED(3) That's the correct order for CUST-AAA. If events were on different partitions, we might see SETTLED before CREATED. Business logic breaks."

Demo 2 - Key partitioning:

"Watch the output. CUST-AAA always → same partition number. That's the hash(key) % numPartitions formula in action."

Demo 3 - Custom partitioner:

"VIP-PLATINUM → Partition 0. Always. Dedicated consumer processes VIP payments first. This is how you implement business priority in Kafka."

Design discussion:

*"What key should our payment system use? paymentId → most uniform distribution, per-payment ordering
customerId → per-customer ordering, risk of hot partitions merchantId → per-merchant ordering, HUGE hot
partition risk (Amazon!)"*

DAY 2 PLAYBOOK

MODULE 1: Schema & Compatibility (60 minutes)

Learning Objectives

- Understand why schemas matter
 - Use Schema Registry with Avro
 - Test schema compatibility
 - Manage schema evolution safely
-

Sub-Module 1.1: Schema Registry Setup (10 min)

Verify Schema Registry:

```
curl http://localhost:8081/subjects
```

Expected: `[]` (empty initially)

Show the Schema Registry UI:

- Open <http://localhost:8081> in browser
- Show the REST API endpoints

Trainer Script:

"Schema Registry is the CONTRACT system for events. Just like Swagger/OpenAPI is the contract for REST APIs, Schema Registry is the contract for Kafka messages.

Without it: Producer changes the schema, all consumers BREAK silently. You discover it at 3am when fraud alerts stop firing."

Sub-Module 1.2: Schema Registry Demo (20 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module1_Schema.SchemaRegistryDemo"
```

Demo 2 - Register V1:

"Schema ID 1. Every message produced with this schema will embed schema ID 1 in its first 5 bytes. Consumers look up ID 1 from registry to deserialize."

Demo 3 - Compatibility Test:

"COMPATIBLE! V2 has new fields but ALL have defaults. Old consumers reading V2 messages use the defaults."

Demo 5 - Breaking Change:

"Schema Registry REJECTED our breaking change! This is the safety net. Without registry, this would deploy to production and break production consumers silently."

Show the schema in UI:

- Navigate to <http://localhost:8081/subjects>
 - Show subject: `payment-created-value`
 - Show versions: `/subjects/payment-created-value/versions`
-

Sub-Module 1.3: Avro Producer (15 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module1_Schema.AvroProducer"
```

Key points:

"See how *GenericRecord* fields map to the Avro schema exactly. Type safety: putting wrong type causes exception at *PRODUCE TIME* not at consume time."

Wire format visualization:

"Every Avro Kafka message: `[0x00][ID-4bytes][avro-binary]` 5 bytes overhead per message. Schema stored *ONCE* in registry. For 1M messages/day: save $800 \text{ bytes} \times 1M = 800MB$ per day!"

Sub-Module 1.4: Avro Consumer + Evolution (15 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module1_Schema.AvroConsumer"
```

Point out in output:

- V1 messages: `processingRegion = N/A`
- V2 messages: `processingRegion = EU-WEST`
- SAME consumer handles BOTH seamlessly

Schema evolution Q&A:

- "How do we rename a field?" → Add new name as alias
 - "How do we remove a field?" → Add a default first, then remove in next version
 - "What compatibility mode for payments?" → FULL (safest)
-

MODULE 2: Kafka Security (60 minutes)

Learning Objectives

- Configure SSL/TLS encryption
 - Set up mutual TLS authentication
 - Design ACL strategy
 - Apply security for financial data
-

Sub-Module 2.1: SSL Concepts (15 min)

Whiteboard - Draw the security layers:

```
[Payment Producer]
|
| TLS 1.3 encrypted tunnel
| Client cert presented (mTLS)
▼
[Kafka Broker :9096]
|
| ACL check: "Can this client write to payment-created?"
▼
[payment-created topic] (data encrypted at rest)
```

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module2_Security.SSLProducerDemo"
```

Sub-Module 2.2: Certificate Generation (10 min)

If certs not generated:

```
cd docker\ssl
generate-certs.bat
```

Explain the cert flow:

"We created a CA (Certificate Authority) - the trusted root. The CA signed the broker cert and client cert. Both sides trust the CA → both trust each other."

Sub-Module 2.3: SSL Producer & Consumer (20 min)

Start SSL broker:

```
docker-compose -f docker\docker-compose-ssl.yml up -d
```

Run SSL Producer:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module2_Security.SSLProducerDemo"
```

Run SSL Consumer:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module2_Security.SSLConsumerDemo"
```

Key points from output:

- SSLProducerDemo Demo 4: "Without SSL" - show what an attacker sees
- "Wireshark shows raw payment JSON without SSL!"
- "Wireshark shows only encrypted bytes with SSL!"

Sub-Module 2.4: ACLs Discussion (15 min)

Walk through ACL design in SSLConsumerDemo output:

"Payment service: WRITE to payment-created ONLY Fraud service: READ payment-created, WRITE fraud-result No service has unnecessary permissions!"

Show ACL CLI commands from output:

```
docker exec kafka-broker-1 kafka-acls \  
  --bootstrap-server localhost:9092 \  
  --add \  
  --allow-principal 'User:CN=payment-service' \  
  --operation Write \  
  --topic payment-created
```

Security Architecture review:

Walk through the 6 security layers from the demo output. Ask students: "Which layer do you currently implement?"

MODULE 3: Event-Driven Architecture (60 minutes)

Learning Objectives

- Distinguish events from commands
 - Design pub-sub systems
 - Choose choreography vs orchestration
 - Design Saga for payments
-

Sub-Module 3.1: Events vs Commands (15 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module3_EDA.PaymentEventDemo"
```

Demo 1 output - Events vs Commands:

"Read the 'PROBLEM' section. These are REAL problems in microservice systems using HTTP calls.

Then read the 'BENEFIT' section - that's what EDA gives us."

Interactive question:

"In your current system, how does PaymentService communicate with FraudService? HTTP? What happens if FraudService is down?" (Traditional: payment fails. EDA: payment queued, processed when fraud service recovers)

Sub-Module 3.2: Pub-Sub Demo (15 min)

Watch Demo 2 - PubSub:

"Three services all subscribed BEFORE the publisher starts. When PaymentService publishes ONE event, all 3 receive it. PaymentService code has NO reference to FraudService, AuditService, or AnalyticsService!"

Show in Kafka UI:

- Consumer Groups: show 3 different groups

- All consuming from payment-created
 - Each has independent offset tracking
-

Sub-Module 3.3: Choreography Pattern (15 min)

Walk through Demo 3 output:

"Each service knows ONLY about its own topic. FraudService knows about payment-created and fraud-result. Nothing else. No URLs. No service discovery."

Draw the event chain:

```
payment-created → FraudService → fraud-result
                        ↓
                SettlementService → payment-settled
                        ↓
                    LedgerService
```

Sub-Module 3.4: Saga Pattern (15 min)

Run Demo:

```
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module3_EDA.SagaPatternDemo"
```

Demo 2 - Happy Path:

"Watch the event chain: CREATED → APPROVED → SETTLED. Each is a separate Kafka message. Each service is autonomous."

Demo 3 - Compensating Transaction:

"Payment of \$99,999 triggers fraud rejection. FraudService publishes FraudRejected. No settlement happens. Event goes to DLQ. The saga COMPENSATED by stopping the flow."

Discussion:

"When would you use Orchestration instead? Complex multi-step flows like: authorize → ship → bill → close Where compensation logic is complex (partial shipments, etc.)"

MODULE 4: EDA Implementation Lab (60 minutes)

Learning Objectives

- Build end-to-end multi-service pipeline
 - Handle failures and DLQ
 - Monitor with Kafka UI
-

Sub-Module 4.1: Pipeline Architecture Review (10 min)

Walk through PipelineOrchestrator output header:

"Three independent Java services. They communicate ONLY through Kafka topics. No shared database. No HTTP calls between them. This is true microservice independence!"

Sub-Module 4.2: Run Full Pipeline (30 min)

Start the pipeline:

```
mvn exec:java -
Dexec.mainClass="com.kafka.training.Day2Module4_Pipeline.PipelineOrchestrator"
```

What to observe (guide students through):

First 10 seconds:

"Watch *PaymentService* log first. Payments generated every 2 seconds. Each has a unique *paymentId* and *correlationId* for tracing."

After 10 seconds:

"*FraudService* kicks in! Watch it consume *PaymentCreated* events. See the risk scores: low for normal amounts, high for \$50k+ payments."

After 20 seconds:

"*SettlementService* processes *APPROVED* payments. Watch for '⌚ Skipping *REJECTED* payment' - that's the filter in action. See the fee calculation: 1.5% deducted."

Switch to Kafka UI regularly:

1. Topics → payment-created → Messages: Show real-time messages
2. Topics → fraud-result → Messages: Show fraud decisions
3. Topics → payment-settled → Messages: Show settlement events
4. Topics → payment-dlq → Messages: Show failures
5. Consumer Groups: Show lag for each group

Sub-Module 4.3: Demonstrate Resilience (10 min)

While pipeline is running:

Test 1: Kill FraudService (Ctrl+C in fraud terminal if running separately)

"*PaymentService* keeps producing. Messages accumulate in *payment-created*. When *FraudService* comes back, it catches up from where it left off. Zero message loss. Consumer group lag shows backlog."

Test 2: Check consumer group lag:

```
scripts\check-consumer-groups.bat
```

"*CURRENT-OFFSET* vs *LOG-END-OFFSET* = *LAG*. Lag tells us how far behind each service is. Production alert: if lag > threshold, scale up consumers!"

Sub-Module 4.4: Pipeline Discussion (10 min)

Discussion questions:

1. "How would you add a *NotificationService* to this pipeline?" → Subscribe to *fraud-result* topic with new consumer group → Zero changes to existing services!

2. "What if Settlement takes 5 seconds instead of 150ms?" → Scale SettlementService from 1 to 3 instances → Each instance gets 1 partition (3 partitions = 3 consumers max) → Or increase partitions for more parallelism
 3. "How would you handle regulatory requirements to store all events for 7 years?" → Set topic retention:
`kafka-configs --alter --entity-type topics --entity-name payment-created --add-config retention.ms=220903200000` → Or use Kafka Connect to archive to S3/cold storage
 4. "How do you trace a payment through the entire pipeline?" → correlationId header is propagated through ALL events → Use distributed tracing (Jaeger/Zipkin) with Kafka headers
-

Day 2 Wrap-Up (15 min)

Key Takeaways Review

Day 1:

- Kafka is a distributed log, not a message queue
- Partitions = parallelism, Replication = fault tolerance
- Producers: acks=all + idempotence for payments
- Consumers: manual commit, consumer groups for scaling
- Ordering: guaranteed within partition via keys

Day 2:

- Schema Registry: contract enforcement, prevents breaking changes
 - SSL/TLS: encrypt in transit, mTLS for authentication
 - ACLs: least-privilege access to topics
 - EDA: events not commands, loose coupling
 - Saga: distributed transactions without 2PC
 - Pipeline: independent services, DLQ for failures
-

Common Troubleshooting During Training

"Cannot connect to Kafka"

```
# Check all containers are running
docker ps

# Restart if needed
scripts\stop-kafka.bat
scripts\start-kafka.bat
```

"Topic not found"

```
scripts\create-topics.bat
```

"Schema Registry not responding"

```
# Check container
docker logs schema-registry

# Restart
docker restart schema-registry
```

"Maven build fails"

```
# Clean and retry
mvn clean package -DskipTests

# Check Java version
java -version # Must be 17
```

"Avro demo fails - schema not found"

```
# First register schema via SchemaRegistryDemo
mvn exec:java -Dexec.mainClass="com.kafka.training.Day2Module1_Schema.SchemaRegistryDemo"
# Then run AvroProducer
```

"SSL demo - certificates not found"

```
cd docker\ssl
generate-certs.bat
# Then start SSL compose
docker-compose -f docker\docker-compose-ssl.yml up -d
```

Quick Reference - CLI Commands

```
# List topics
docker exec kafka-broker-1 kafka-topics --bootstrap-server localhost:9092 --list

# Describe topic
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server localhost:9092 --describe --topic payment-created

# Console producer (manual messages)
docker exec -it kafka-broker-1 kafka-console-producer \
  --bootstrap-server localhost:9092 --topic payment-created

# Console consumer (from beginning)
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic payment-created \
  --from-beginning --max-messages 10

# Consumer group lag
```

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server localhost:9092 --describe --group fraud-detection-service-group

# List consumer groups
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server localhost:9092 --list

# Schema Registry - list subjects
curl http://localhost:8081/subjects

# Schema Registry - get latest schema
curl http://localhost:8081/subjects/payment-created-value/versions/latest
```

Timing Guide

Module	Allotted	Actual Demo	Discussion	Buffer
D1-M1 Why Kafka	60 min	20 min	25 min	15 min
D1-M2 Architecture	60 min	25 min	20 min	15 min
D1-M3 Prod/Cons	60 min	30 min	20 min	10 min
D1-M4 Reliability	60 min	25 min	25 min	10 min
D2-M1 Schema	60 min	25 min	20 min	15 min
D2-M2 Security	60 min	20 min	25 min	15 min
D2-M3 EDA	60 min	25 min	25 min	10 min
D2-M4 Pipeline	60 min	35 min	15 min	10 min

Final Project Structure Summary

```
kafka-training/
├─ docker/
│   ├── docker-compose.yml      ← Main cluster (Day 1 + Day 2)
│   ├── docker-compose-ssl.yml  ← SSL cluster (Day 2 M2)
│   └─ ssl/
│       └─ generate-certs.bat    ← Certificate generation
├─ scripts/
│   ├── setup.bat                ← Environment check
│   ├── start-kafka.bat          ← Start cluster
│   ├── stop-kafka.bat           ← Stop cluster
│   ├── create-topics.bat        ← Create all topics
│   ├── produce-events.bat       ← CLI producer demo
│   ├── consume-events.bat       ← CLI consumer demo
│   └─ check-consumer-groups.bat ← Lag monitoring
├─ src/main/
└─ java/com/kafka/training/
```



```
| | |─ Day1Module1_WhyKafka/
| | |─ Day1Module2_Architecture/
| | |─ Day1Module3_ProducersConsumers/
| | |─ Day1Module4_Reliability/
| | |─ Day2Module1_Schema/
| | |─ Day2Module2_Security/
| | |─ Day2Module3_EDA/
| | └─ Day2Module4_Pipeline/
| └─ resources/
|   │─ avro/                ← Schema files
|   │─ logback.xml          ← Logging config
|   └─ application.properties
└─ pom.xml
└─ README.md
└─ TRAINER_PLAYBOOK.md
```

The project is fully complete. Every module has working code ready to run on **Windows 11 + Java 17 + Docker Desktop** with `mvn exec:java` commands. The **Trainer Playbook** provides step-by-step guidance, scripts, timing, and troubleshooting for each sub-module across both days.